

CCE-321 Solved Answer Guide

Complete Solved Answer Guide — Based on 11 Past Papers (2014–2023)

[How to Use This Guide](#)

[1. Memory \(RAM/ROM\) Interfacing with Microprocessor ★★★](#)

[2. LVDT / RVDT ★★★](#)

[3. Sensors, Transducers & Transducer Theory ★★★](#)

[4. 8255 / 82C55 Programmable Peripheral Interface \(PPI\) ★★★](#)

[5. Display Systems: CRT, LCD, Color, Scan Types ★★★](#)

[6. Internet of Things \(IoT\) ★★★](#)

[7. Keyboards & Interfacing ★★★](#)

[8. 8279 Programmable Keyboard/Display Controller ★★★](#)

[9. Printers ★★★](#)

[10. USB & Serial/Parallel Buses ★★★](#)

[11. Number Systems & Floating Point ★★](#)

[12. Serial Communication / RS-232 / 8251A ★★](#)

[13. Coding & Interrupts ★★](#)

[14. Floppy Disk Controller & Encoding ★★](#)

[15. Human Computer Interaction \(HCI\) ★★](#)

[16. Storage Interfaces: RAID, SCSI, ATA ★★](#)

[17. Buffers, Spoolers & Interfaces ★★](#)

[18. Wireless Sensor Networks, Real-Time Systems & Misc. ★](#)

[Final Notes](#)

How to Use This Guide

This guide answers every question in the *CCE-321 Important Question Analysis* document, grouped by topic exactly as in the original bank. Priority markers (★★★, ★★, ★) are carried over so you can see what to study first. Diagrams are described in words (construction/block layout) since this is a text answer key — sketch them out on paper while reading, as most exams award marks for a clearly labelled diagram.

1. Memory (RAM/ROM) Interfacing with Microprocessor ★★★

Background you need for this whole section

A microprocessor talks to a memory chip using three groups of lines:

- **Address lines (A0–An):** select one location among many inside a chip, and (via decoding) select *which chip* is being addressed.
- **Data lines (D0–D7 for 8-bit systems):** carry the actual data.
- **Control lines:** RD (read), WR (write), and CS/CE (chip select).

Two decoding styles are used:

- **Linear (partial) decoding:** a few high-order address lines are used directly (or through simple gates) to drive CS. Cheap and simple, but wastes address space because unused/“don’t care” address bits cause the chip to respond at multiple addresses (memory foldback).
- **Full (absolute) decoding:** every unused high-order address line is decoded through a decoder (e.g., 74LS138) so each chip occupies one unique, contiguous block of addresses with no foldback.

General method for every “interface a chip to a given address” question: 1. Find the chip’s capacity (e.g., $4K \times 8$) and work out how many address lines it needs internally: $4K = 2^{12}$, so it needs A0–A11 on-chip. 2. The remaining higher-order address lines (A12, A13, ... up to the processor’s address width) are used for chip selection. 3. Decide linear or full decoding as asked. Draw the decoder/gate feeding CS. 4. Compute the start and end address by fixing the “select” bits to the value that matches the given starting address, then letting the internal address bits range from all-0 to all-1.

Q: Interface two chips of 4K RAM and 1/2 chip(s) of 2K ROM with processor for a given starting address (8000H/9000H/A000H)

Answer approach: - 4K RAM $\Rightarrow$ needs 12 address lines (A0-A11) per chip. Two 4K chips = 8K total, needing one extra line (A12) to distinguish chip 1 from chip 2. - 2K ROM $\Rightarrow$ needs 11 address lines (A0-A10) per chip. - Take a 16-bit address bus (A0-A15) as typical for this course. - For starting address **8000H** = 1000 0000 0000 0000: fix A15=1, A14=0, A13=0, A12 = 0 for RAM chip 1 (address 8000H-8FFFH) and A12 = 1 for RAM chip 2 (address 9000H-9FFFH). Feed A15, A14, A13 (and A12 where needed) into a decoder (e.g., 3-to-8 line 74LS138) whose outputs go to the CS pins of the RAM chips; A0-A11 go directly to the chip's own address pins. - For the ROM chip(s) starting at, say, A000H = 1010 0000 0000 0000: A15=1, A14=0, A13=1, A12=0 selects the block A000H-A7FFFH (2K, since only A0-A10 vary). If two ROM chips are required, A11 differentiates them, giving A000H-A7FFFH and A800H-AFFFFH. - Draw one diagram: Processor $\rightarrow$ Address bus splits into (a) low-order lines straight to each chip's address pins, (b) high-order lines into a decoder $\rightarrow$ decoder outputs into each chip's CS pin. RD/WR from the processor go to every chip's OE/WE pins (ROM only needs OE). - **Memory map table** (example for 8000H start, two 4K RAM chips):

Chip	CS condition	Address Range
RAM1	A12=0	8000H-8FFFH
RAM2	A12=1	9000H-9FFFH

Q: Connect the microprocessor to a given ROM/RAM chip (A0-A9, CS, RD/WR shown) to realize 0000H-03FFFH or 0000H-0FFFFH

- A0-A9 = 10 lines $\Rightarrow$ chip capacity = 2^{10} = 1K locations.
- 0000H-03FFFH is exactly 1K (03FFFH – 0000H + 1 = 400H = 1024), so this chip is mapped starting at address 0000H with the remaining higher address lines (A10 upward) all held at 0 (via a NAND/NOR gate or decoder) to generate CS = active only when those bits are 0.
- For 0000H-0FFFFH (4K range) using a chip that only has A0-A9 (1K chip), you would need **four** such chips, with A10 and A11 used to distinguish the 4 blocks (00000-03FFF, 0400-07FFF, 0800-0BFFF, 0C00-0FFFF), each block's CS generated by a 2-to-4 decoder driven by A10, A11.
- Diagram: A0-A9 $\rightarrow$ chip address pins directly; A10, A11 (and A12+ tied low through a NOR gate) $\rightarrow$ decoder $\rightarrow$ CS of each of the 4 chips; RD $\rightarrow$ OE, WR $\rightarrow$ WE (RAM only).

Q: Illustrate an 8-bit microprocessor interfaced to a 4K RAM system using full/partial decoding

- 4K needs A0-A11 (12 lines).
- **Partial/linear decoding:** Use only, say, A15 directly as CS (active low, so CS = NOT A15). This makes the chip respond whenever A15 = 0, *regardless* of A12-A14, causing the same 4K of physical memory to appear repeated (foldback) at 0000H, 1000H, 2000H, 3000H, ... up to 7000H (any address where A15=0). Cheap (no decoder chip) but wastes address space.
- **Full decoding:** Feed A12-A15 into a 4-to-16 (or combination of gates/74LS138 decoders) so CS is asserted for exactly one 4K block, e.g., only when A12-A15 = 0000, giving a unique range 0000H-0FFFFH with no repeats elsewhere.
- Answer should state: partial decoding $\rightarrow$ simple circuit, multiple (aliased) addresses map to the same physical byte; full decoding $\rightarrow$ more logic gates but a clean, unique 4K window.

Q: Design a 1K $\times$ 8 RAM system using a 512 $\times$ 8 RAM chip (CS1 = L, CS2 = H)

- 512 $\times$ 8 = $2^9 \times 8$, so each chip uses A0-A8 (9 lines) and needs 2 chips to reach 1K (2^{10}).
- A9 is the extra address line: when A9 = 0, select Chip 1; when A9 = 1, select Chip 2.
- Chip 1: CS1 must be Low and CS2 High to enable it $\rightarrow$ connect CS1 = A9 (inverted, i.e., NOT A9) if CS1 is active-low, and tie CS2 = 0 (ground, since CS2 must be High to enable — check the chip's true polarity from its datasheet symbol; typically CS1 active-low needs an inverter fed from A9, CS2 active-high is tied directly to A9's complement or a fixed logic level matching the required "enable" state).
- Practically: Chip 1 enabled when A9 = 0 $\rightarrow$ CS1 = A9' (active low, so CS1 asserted when A9=0 works if CS1 is inverted A9), CS2 tied to a level that is always "H" as required. Chip 2 enabled when A9 = 1 $\rightarrow$ its CS1 = A9 (through inverter as needed), CS2 fixed appropriately.
- **Memory map (linear decoding, don't-cares = 1):** Chip 1: A9=0, A0-A8 = don't care $\rightarrow$ 0000 0000 00 to 0111 1111 11 in binary = 000H-1FFFH. Chip 2: A9=1 $\rightarrow$ 200H-3FFFH. (Exact hex boundary depends on total address width chosen; the key marks are for correctly splitting the 1K range into two contiguous 512-byte halves using A9.)

Q: 14-bit address, 8-bit word — capacity, chips needed, chip-select bits

- 14-bit address $\Rightarrow 2^{14}$ = **16,384 bytes = 16 KB** can be stored (8-bit word size just means each location stores 1 byte).
- If built from 1K $\times$ 1 RAM chips (2^{10} locations $\times$ 1 bit each):
 - To get 8-bit width, you need 8 chips *in parallel* per 1K block (one chip per data bit).
 - To get 16K total depth from 1K chips, you need 16K/1K = 16 such "rows" of chips.
 - Total chips = 16×8 = **128 chips**.
- Chip select bits: 16 rows need $\log_2(16)$ = **4 bits** for chip select (decoded from the top 4 address lines, A10-A13), while A0-A9 go to every chip's address pins.

Q: 16-bit address, byte addressable — interface 4096 $\times$ 8 RAM

- Memory capacity of the RAM = $4096 \times 8 = 4\text{K}$ bytes, needing A0–A11 (12 lines).
- Processor has 16 address lines (64K address space), so 4 lines (A12–A15) remain for chip selection.
- Number of RAM chips: since one 4096×8 chip already gives the full requested capacity, **only 1 chip** is required (unless the question wants you to fill the entire 64K space, in which case $64\text{K}/4\text{K} = 16$ chips and a 4-to-16 decoder, e.g., built from two 74LS138s, is needed for the address decoder using A12–A15).
- Size of address decoder: needs to decode 4 lines $\rightarrow$ a 4-to-16 line decoder (or 2×3 -to-8 decoders combined).

Q: Standard I/O vs Memory-mapped I/O

Aspect	Standard (Isolated) I/O	Memory-Mapped I/O
Address space	Separate I/O address space (e.g., 8-bit, 256 ports in 8085)	Shares the same address space as memory
Instructions	Special IN/OUT instructions	Any memory-reference instruction (MOV, ADD, etc.) can access I/O
Control signals	Uses IO/M, RD, WR distinct from memory read/write	Uses the same MEMR/MEMW control lines
Address lines used	Fewer lines usually needed for I/O	Uses full address bus, so I/O devices consume part of memory address space
Speed/flexibility	Slightly faster dedicated access	More flexible (arithmetic/logic ops directly on I/O data) but reduces available memory addresses

Q: ISA bus vs PCI bus

- **ISA (Industry Standard Architecture):** older 8/16-bit parallel bus, low clock speed (~ 8 MHz), processor-dependent timing, no plug-and-play, lower throughput — used for slow peripherals (old sound/network cards).
- **PCI (Peripheral Component Interconnect):** 32/64-bit bus, higher clock (33/66 MHz+), processor-independent, supports plug-and-play and bus mastering (DMA-capable), much higher data-transfer rate, and automatic device configuration.

Q: Methods used in main memory array design (and pros/cons)

- **Linear select (fully decoded) method:** each word has a dedicated select line from a decoder; simple wiring for small arrays but decoder size grows quickly for large memories (needs 2^n outputs for n address bits) — not scalable.
- **2-D (coincident) decoding array:** address split into row and column decoders; a cell/word is selected only when both its row and column lines are active. Greatly reduces the number of decoder outputs needed ($\sqrt{N} + \sqrt{N}$ instead of N), so it scales much better, at the cost of slightly more complex addressing logic and needing both X and Y decoders to agree.
- **2.5-D / word-organized approach** (sometimes discussed): a compromise using linear selection for words but 2-D addressing internally for bit cells — advantage is faster access, disadvantage is more complex chip layout. In short: linear decoding = simple but not scalable; 2-D decoding = scalable and used in real large RAM/ROM chips, slightly more complex control.

2. LVDT / RVDT ★★★

Q: What is an LVDT (or RVDT)? How does it work step by step? Explain with construction diagram.

LVDT (Linear Variable Differential Transformer) is a passive inductive transducer that converts linear displacement into a proportional AC electrical output.

Construction: - One **primary winding** at the centre of a cylindrical former, and two **secondary windings** wound identically on either side of the primary, connected in **series opposition** (so their EMFs oppose each other). - A **ferromagnetic core** (soft iron), free to move along the axis of the former, links the primary flux to the two secondaries.

Working, step by step: 1. An AC excitation voltage (typically a few kHz) is applied to the primary winding, producing an alternating magnetic flux. 2. This flux links to both secondary windings and induces EMFs (E_1 in secondary 1, E_2 in secondary 2). 3. When the core is exactly at the **centre (null position)**, flux linkage to both secondaries is equal, so $E_1 = E_2$, and because the windings are connected in opposition, the net output voltage $E_{\text{out}} = E_1 - E_2 = 0$. 4. When the core moves toward secondary 1 (say, to the right), more flux links secondary 1 and less links secondary 2, so $E_1 > E_2$, giving a net output proportional to displacement, in phase with the primary voltage. 5. When the core moves to the left (toward secondary 2), $E_2 > E_1$, and the output has the same magnitude relationship but is now 180 degrees out of phase with respect to the previous case. 6. The magnitude of E_{out} is directly proportional to the amount of core displacement, and the phase indicates the direction. A phase-sensitive demodulator circuit converts this AC signal into a DC voltage proportional to displacement (with sign indicating direction).

RVDT (Rotary Variable Differential Transformer) works on the exact same differential-transformer principle, except the core is cam-shaped and rotates about a pivot instead of sliding linearly, so the output is proportional to angular displacement (rotation) rather than linear displacement. Everything else (primary/secondary windings, null position, phase-sensitive detection) is identical in principle.

Advantages: frictionless (no physical contact between core and windings, long life, no wear), high sensitivity and resolution, good linearity over its rated range, infinite resolution (theoretically), robust and unaffected by moderate temperature/humidity variation.

Q: Effect of variation of dielectric constant on a capacitive transducer measuring displacement

A capacitive transducer's capacitance is $C = \epsilon A/d$ (ϵ = permittivity, A = plate area, d = plate separation). If displacement is measured by moving a dielectric slab in/out of the gap between fixed plates, the effective permittivity changes as the dielectric material's overlap area with the plates changes. Since C is directly proportional to the dielectric constant, sliding the dielectric increases C proportionally to the length inserted, giving a linear relationship between displacement and capacitance. This configuration is preferred for measuring larger displacements linearly, whereas varying the plate separation (d) gives a non-linear ($1/d$) response usable mainly for very small displacements.

Q: How does a capacitive sensor operate to measure linear displacement? Applications of non-contact sensors.

- A capacitive displacement sensor typically uses one fixed plate and one plate attached to the moving object, with air as the medium between them: $C = \epsilon_0 \cdot \epsilon_r \cdot A/d$.
- As the object moves, d (the plate gap) changes, changing C inversely with distance; this change in C is measured by an AC bridge or oscillator circuit whose frequency/amplitude shift is calibrated against displacement.
- Alternatively, area of overlap (A) can be varied for a more linear response over larger ranges.
- Applications where non-contact sensing is preferred: measuring vibration or thickness of delicate or rotating surfaces (thin film thickness, rotating shaft run-out, semiconductor wafer flatness) where physical contact would damage the surface or introduce friction/loading errors; also hazardous, very high-speed, or high-temperature environments.

3. Sensors, Transducers & Transducer Theory ★★★

Q: Main difference between Sensors and Transducers

- A sensor is a device that detects/responds to a physical stimulus (heat, light, pressure, motion) and produces a signal representing information about that stimulus, primarily about sensing/detecting a quantity.
- A transducer is a broader device that converts energy from one form to another, most commonly converting a non-electrical physical quantity into an electrical signal (or vice versa). Every sensor that outputs an electrical signal is technically a transducer, but a transducer need not sense anything intelligently, e.g., a loudspeaker (electrical to sound) is a transducer but not a sensor.
- In short: all sensors that give electrical output are transducers, but not all transducers are sensors; sensing implies measurement/detection intent, while transduction is purely about energy-form conversion.

Q: Define transducers; advantages of electrical signal as transducer output

Transducer: a device that converts one form of energy (mechanical, thermal, optical) into another, usually into an electrical signal proportional to the measured quantity.

Advantages of electrical output: easy to amplify, filter, and condition using standard electronic circuits; can be transmitted over long distances with minimal loss; easily digitized (via ADC) for processing, storage, and display by computers/microprocessors; fast response and high accuracy achievable; can be easily recorded, multiplexed, and used in feedback/control systems.

Q: Active vs passive transducers; how sensors and transducers differ in function/application

- **Active transducers** generate their own output signal directly from the input physical quantity, without needing an external power source, e.g., thermocouple (generates EMF from heat via the Seebeck effect), piezoelectric crystal (generates voltage from applied pressure).
- **Passive transducers** require an external power/excitation source to produce an output; they typically change some electrical parameter (resistance, capacitance, inductance) in response to the input, which is then measured using an external circuit, e.g., LVDT, strain gauge, thermistor.
- Sensors are chosen/described in terms of what physical quantity they detect, while transducer classification (active/passive) describes how they convert that quantity into a usable signal; sensors are the front end of measurement systems while transducers are the general conversion mechanism that may be embedded inside a sensor or actuator.

Q: Nonlinearity error vs Hysteresis error

- **Nonlinearity error:** the maximum deviation of the actual input-output calibration curve from an ideal straight line, expressed as a percentage of full-scale output, because most real transducers do not have a perfectly linear response over their entire range.
- **Hysteresis error:** the difference in output for the same input value depending on whether that input was reached while increasing or decreasing the input, caused by energy storage/dissipation effects (friction, elastic deformation, magnetic domain lag).
- **Key difference:** nonlinearity is about the shape of the input-output curve deviating from a straight line; hysteresis is about the same input producing different outputs depending on direction of approach (a path-dependent/memory effect).

Q: Piezo-electric effect and voltage sensitivity derivation

Piezoelectric effect: certain crystalline materials (quartz, PZT, Rochelle salt) generate an electric charge/voltage across their faces when mechanical stress is applied, and conversely deform when an electric field is applied.

Derivation: Applied force F on a crystal of area A produces stress $= F/A$. The generated charge Q is proportional to the applied force: $Q = dF$, where d is the piezoelectric charge constant (C/N). The crystal behaves like a capacitor of capacitance $C_p = eA/t$ (t = thickness), so the open-circuit voltage produced is $V = Q/C_p = (dFt)/(eA)$. **Voltage sensitivity $g = V/(F/A) = d*t/e$ per unit stress**, showing voltage output is proportional to applied force and thickness, and inversely proportional to permittivity and electrode area.

Q: Short notes: photoemissive, photoconductive, photovoltaic cells

- **Photoemissive cell:** works on the photoelectric effect; incident light striking a photosensitive cathode inside a vacuum/gas tube emits electrons collected by an anode, producing current proportional to light intensity.
- **Photoconductive cell (LDR):** resistance of a semiconductor (e.g., cadmium sulfide) decreases as light intensity increases. Requires external voltage; used in light-dependent switches, exposure meters.
- **Photovoltaic cell (solar cell):** a p-n junction that directly generates a voltage when light falls on it, no external bias needed; used in solar panels and self-powered light meters.

Q: Hall effect and Hall-effect transducers

Hall effect: when a current-carrying conductor/semiconductor is placed in a magnetic field perpendicular to the current, a voltage (Hall voltage V_H) develops across it, perpendicular to both current and field, due to the Lorentz force deflecting charge carriers. $V_H = (IB)/(ne*t)$.

Hall-effect transducers: use this principle to measure magnetic field strength directly, or, by keeping current constant and letting a moving magnet vary B , to sense position, displacement, proximity, or rotational speed non-contact (e.g., brushless motor commutation, proximity switches, current sensors).

Q: Thermistor applications and limitations

Applications: temperature measurement and compensation, over-current/over-temperature protection, liquid-level detection, time-delay devices, inrush-current limiting.

Limitations: highly non-linear resistance-temperature characteristic; limited temperature range compared to thermocouples/RTDs; self-heating error; fragile construction in some types; ageing/drift over time.

Q: Pyrometer definition, radiation pyrometer principle, advantages/limitations

Pyrometer: an instrument for measuring high temperatures of surfaces that cannot be touched, by sensing emitted thermal radiation, a non-contact temperature sensor.

Working principle: emitted radiation is collected by a lens/mirror and focused onto a detector (thermopile, photodiode, bolometer); output is proportional to received radiant energy which, by Stefan-Boltzmann's law (power proportional to T^4), is calibrated to indicate temperature.

Advantages: no contact required, fast response, does not disturb the object's temperature field.

Limitations: accuracy depends on knowing the target's emissivity; affected by intervening atmosphere (dust, smoke, humidity); generally more expensive/less accurate than contact methods at lower temperatures.

Q: Thermocouple definition, construction, operating principle

Thermocouple: a temperature sensor made of two dissimilar metal wires joined at one end (hot/measuring junction), based on the Seebeck effect.

Construction: two wires of different metals (e.g., Chromel-Alumel for Type K) welded/twisted at the measuring junction; open ends form the reference/cold junction connected to the measuring instrument.

Operating principle: when the two junctions are at different temperatures, a small EMF is generated proportional to the temperature difference (Seebeck effect); measured and, knowing the reference temperature, converted into the hot-junction temperature using standard tables.

Q: Short notes: U-type and Well-type manometer

- **U-tube manometer:** a U-shaped tube partially filled with liquid (mercury, water, oil); one limb connects to the pressure source, and the level difference between the two limbs gives $p = \rho gh$.
- **Well-type (cistern) manometer:** one limb is replaced by a much larger reservoir; because its cross-section is far larger, the level drop in the well is negligible, so essentially all level change appears in the single narrow tube, requiring only one reading instead of two, at the cost of a small area-ratio calibration correction.

Q: Bellows vs capsules sensitivity (True/False with justification)

True. Bellows are more sensitive than capsules. A bellows has a series of deep convolutions (folds) giving much larger effective area and greater axial deflection per unit pressure than a capsule (a shallow diaphragm chamber with limited travel), so bellows give greater displacement per unit pressure change, though capsules tend to be more accurate/linear over a narrower range and more compact.

4. 8255 / 82C55 Programmable Peripheral Interface (PPI) ★★★

Q: Pin configuration of 8255 PPI and operation of each unit

The 8255 is a 40-pin general-purpose I/O chip providing 24 programmable I/O lines organized as three 8-bit ports: **Port A, Port B, Port C** (Port C can be split into two 4-bit halves, C-upper and C-lower).

Pin groups: - **D0-D7:** bidirectional data bus, connects to the microprocessor's data bus. - **RD, WR:** read/write control signals from the processor. - **CS (Chip Select):** enables the chip. - **A0, A1:** address lines used with CS to select one of Port A, Port B, Port C, or the Control register internally. - **RESET:** clears the control register (all ports default to input mode). - **PA0-PA7, PB0-PB7, PC0-PC7:** the actual I/O port pins connected to external peripherals.

Internal functional units: 1. **Data bus buffer:** bidirectional tri-state buffer interfacing the 8255's internal bus to the system data bus. 2. **Read/Write control logic:** manages all internal/external data transfers; it interprets RD, WR, CS, A0, A1, and RESET to route data to/from the correct port or control register. 3. **Group A and Group B control:** Group A controls Port A and the upper half of Port C; Group B controls Port B and the lower half of Port C. Each group receives mode-selection commands from the control word and sets up the ports accordingly (Mode 0, 1, or 2). 4. **Port A, Port B, Port C:** the actual 8-bit latches/buffers that hold data going to or coming from the peripheral devices, individually programmable as input or output.

Q: Define 8155 PPI and its features

The **8155** is a combination chip that provides RAM plus I/O — not purely a PPI, but often studied alongside it. It contains: 256 bytes of static RAM, two 8-bit general-purpose I/O ports (Port A, Port B) and one 6-bit I/O port (Port C), and a 14-bit programmable timer/counter. Features: single +5V supply, multiplexed address/data bus (directly compatible with 8085), ports individually programmable as input/output, on-chip timer usable for real-time interrupts or square-wave generation.

Q: What is 82C55 PPI? Operation of a stepper motor connected with 82C55

The **82C55** is the CMOS (low-power) version of the 8255A PPI, functionally identical to the 8255 but consuming much less power.

Stepper motor operation via 82C55: the motor's 4 phase windings are connected to 4 bits of one output port (e.g., PA0-PA3), each driven through a driver/transistor stage. The processor is programmed to output a specific 4-bit excitation pattern sequence (e.g., 1001, 1010, 0110, 0101 for half-step, or simpler sequences for full-step) to that port at timed intervals; each new pattern energizes a different combination of windings, causing the rotor to advance by one step. Reversing the sequence order reverses the direction of rotation, and the delay between successive output patterns (via software timing loops or a timer) controls motor speed.

Q: Control word for 8255A: Mode 0, Port A output, Port B input, Port C output

The 8255 control word format (Mode 0, Group A and Group B) is: D7=1 (mode set flag), D6D5 (Group A mode, 00 for Mode 0), D4 (Port A: 0=output, 1=input), D3 (Port C upper: 0=output, 1=input), D2 (Group B mode, 0 for Mode 0), D1 (Port B: 0=output, 1=input), D0 (Port C lower: 0=output, 1=input).

Given: Port A = output (D4=0), Port B = input (D1=1), Port C = output (D3=0, D0=0), both groups in Mode 0 (D6D5=00, D2=0):

Control word = 1 00 0 0 0 1 0 = **10000010 = 82H**.

Q: How can we assign input and output ports of 8255?

Ports are assigned as input or output purely by **software**, by writing the appropriate **control word** to the 8255's control register (selected via A1A0 = 11). Each bit of the control word (as detailed above) sets whether Port A, Port B, and the two halves of Port C are inputs or outputs, and which mode (0, 1, or 2) each group operates in. No hardware changes are needed to switch a port's direction — simply issuing a new control-word byte reconfigures the ports at run time.

5. Display Systems: CRT, LCD, Color, Scan Types ★★★

Q: Operation of a delta-delta shadow-mask CRT

In a **delta-delta shadow-mask CRT**, three electron guns are arranged in a triangular (delta) pattern, each aimed at phosphor dots of

one primary color (red, green, blue), also arranged in delta-shaped triads on the screen. A thin **metal shadow mask**, perforated with a matching pattern of small holes, is placed just behind the phosphor screen.

Operation: each of the three electron beams approaches the mask at a slightly different angle (because the guns are physically separated in the delta arrangement); the shadow mask's holes are positioned so that each beam, after passing through a given hole, can only strike the phosphor dot of its own designated color (the mask "shadows"/blocks it from hitting the other two colors' dots). By varying the intensity of each of the three beams (controlled by the video signal for R, G, B), the combination of light from the three adjacent phosphor dots blends (due to the eye's limited resolution) to produce the perceived color at that point on the screen.

Q: Raster scan vs Random scan display — definitions and differences

- **Raster scan display:** the electron beam sweeps across the screen row by row (left to right, top to bottom) in a fixed pattern, similar to how a page of text is read; the image is formed by varying the beam's intensity at each point of this fixed sweep pattern (pixels), and the whole screen is refreshed at a fixed rate (e.g., 60 Hz) regardless of what needs to be drawn.
- **Random scan (vector/calligraphic) display:** the electron beam is directed only along the paths needed to draw the actual line-segments of the picture (like a pen drawing on paper), moving directly from one endpoint to another rather than scanning every point of the screen.

Aspect	Raster Scan	Random Scan
Beam movement	Fixed left-right, top-bottom sweep of entire screen	Only along drawn line segments
Image storage	Frame buffer stores intensity for every pixel	Display file stores line-drawing commands
Realism/shading	Good for realistic, filled, shaded images	Poor for filled areas; best for line drawings
Refresh dependency	Refresh rate independent of picture complexity	Refresh rate depends on number of lines (more lines = more flicker)
Typical use	Modern monitors, TVs	Old vector graphics terminals, oscilloscopes

Q: True color representation, disadvantages, N-bits-per-color color count

True color means each pixel's color is represented using enough bits per RGB channel (typically 8 bits per channel = 24-bit color) that the displayed image can reproduce a range of colors indistinguishable to the human eye from a continuous-tone (real-world) image — commonly 24-bit color, giving $2^{24} \approx 16.7$ million colors.

Disadvantages/problems: requires much larger frame-buffer memory and bandwidth than lower-bit-depth modes; increases processing and storage requirements for images/video; may not be necessary or noticeable on all display hardware, wasting resources.

If N bits per primary color are used: number of possible levels per primary = 2^N , so the total number of simultaneous colors possible (using 3 primaries, R,G,B) = $(2^N)^3 = 2^{(3N)}$.

Q: Short notes — Interlaced refresh, resolution-related terms, Bitmap and Pixmap

- **Interlaced refresh:** instead of refreshing every scan line in a single top-to-bottom pass, the screen is refreshed in two passes — first all odd-numbered lines, then all even-numbered lines — effectively doubling the perceived refresh rate for a given bandwidth and reducing flicker on CRTs, at the cost of possible line-flicker/artifacts on high-contrast or fast-moving content.
- **Bitmap:** a data structure representing an image where each bit (or a small fixed set of bits) corresponds to one pixel, typically used for monochrome (1 bit per pixel) images.
- **Pixmap:** similar to a bitmap but with multiple bits per pixel, allowing each pixel to represent a range of colors or gray levels (used for color images).

Q: LCD advantages/disadvantages over CRT

Advantages of LCD: much thinner and lighter; consumes far less power; produces less heat; no geometric/edge distortion (flat panel, no electron-beam deflection issues); no flicker at typical refresh rates because pixels stay lit rather than being refreshed by a scanning beam; emits negligible electromagnetic radiation.

Disadvantages of LCD: limited viewing angle (image quality can degrade off-axis, though modern IPS panels reduce this); generally slower pixel response time historically (motion blur/ghosting on fast content, improved in modern panels); fixed native resolution — non-native resolutions may need scaling and look blurrier; typically higher manufacturing cost per unit area of very large screens historically; can suffer from dead/stuck pixels.

Q: Selection criteria of a monitor / what determines monitor quality

Key factors: **resolution** (number of pixels, sharper images at higher resolution), **dot/pixel pitch** (distance between adjacent phosphor dots or sub-pixels — smaller pitch = sharper image), **refresh rate** (higher rate reduces flicker), **screen size and aspect ratio**, **brightness and contrast ratio**, **viewing angle**, **response time** (important for video/gaming), **color accuracy/gamut**, and **connectivity/interface compatibility** with the computer.

Q: Working principle of LCD monitor and component interaction

An **LCD (Liquid Crystal Display)** does not emit its own light (except in some self-emissive OLED-related variants); instead it modulates light from a backlight.

Components and interaction: 1. A **backlight** (LED array typically) provides a constant, uniform white light source behind the panel. 2. Light first passes through a **rear polarizing filter**, which only allows light waves oscillating in one plane to pass. 3. It then passes through a layer of **liquid crystal molecules** sandwiched between two glass/electrode layers; when a voltage is applied to a given pixel's electrodes (via the TFT — thin-film transistor — matrix), the liquid crystal molecules twist/untwist, rotating the plane of polarization of the light passing through by a controllable amount. 4. The light then passes through a **front polarizing filter**, oriented at 90° to the rear one; depending on how much the liquid crystal rotated the light's polarization, more or less light passes through this second filter — this is how each pixel's brightness is controlled. 5. A **color filter layer** (tiny red, green, blue filters over sub-pixels) then imparts color to the transmitted light, and the combination of R, G, B sub-pixel intensities produces the final perceived color at each pixel position.

Q: Serialize the steps to turn a pixel on/off in an LCD monitor

1. The processor/graphics controller determines the desired brightness value for the target pixel from the frame buffer.
2. The row (gate) driver activates the specific row line (via the TFT matrix) corresponding to the pixel's row, turning "on" the thin-film transistor for that row.
3. The column (data) driver applies the corresponding analog voltage level to the column line for that pixel, based on the desired brightness.
4. With the TFT switched on, the applied voltage charges the liquid-crystal cell's capacitor at that pixel location, causing the liquid crystal molecules to twist by an amount proportional to the voltage.
5. This twisting rotates the polarization of light passing through, controlling how much backlight passes through the front polarizer at that pixel (partially "on" = grey level, fully blocked = pixel "off"/black, fully passed = pixel maximum brightness).
6. The row is then deactivated (TFT switched off), and the charge is held on the pixel's storage capacitor until the next refresh cycle, when the whole process repeats for the next frame.

Q: What is a plasma monitor?

A **plasma display** uses small cells filled with an inert ionizable gas (like neon/xenon) sandwiched between glass panels. When a voltage is applied across a cell's electrodes, the gas is ionized into a plasma state, which emits ultraviolet light; this UV light strikes a phosphor coating on the inside of the cell (red, green, or blue phosphor per sub-pixel), causing it to glow visible light, similarly to a fluorescent lamp. Each cell acts as a self-emitting pixel, giving good contrast and wide viewing angles, but historically higher power consumption and risk of screen burn-in compared to LCD/OLED.

Q: Find CMY from RGB; N-bits-per-color possible colors; pitch; why color printers use black pigment

- **RGB to CMY conversion:** $C = 1 - R$, $M = 1 - G$, $Y = 1 - B$ (values normalized to a 0-1 or 0-255 scale, e.g., on a 0-255 scale: $C = 255 - R$, $M = 255 - G$, $Y = 255 - B$). Substitute the given R, G, B values directly into this formula to get C, M, Y.
- **Direct coding, N bits per color, 3 colors:** total possible colors per pixel = $2^{(3N)}$ (same reasoning as the true-color question above).
- **Pitch of a color CRT:** the distance between two adjacent phosphor dots of the *same* color on the screen (dot pitch); smaller pitch gives a sharper image since dots are closer together.
- **Why color printers use black pigment (in addition to CMY):** mixing full amounts of cyan, magenta, and yellow ink together to produce black is imprecise (produces a muddy dark brown rather than a true black) and also consumes far more ink; using a separate black (K) ink/pigment produces sharper text and true blacks more economically — hence the CMYK model.

Q: Cut a 512x512 sub-image from an 800x600 image — pixel coordinates of lower-left corner

To centre a 512x512 sub-image within an 800x600 image, the lower-left corner coordinates are found by centring the smaller image inside the larger one: - Horizontal offset = $(800 - 512)/2 = 144$ - Vertical offset = $(600 - 512)/2 = 44$ So the lower-left corner of the sub-image is at pixel coordinates **(144, 44)**, and it extends to $(144+512, 44+512) = (656, 556)$. (If the question specifies a different corner/anchor than "centred," adjust the offset formula accordingly — the method of subtracting the sub-image dimension from the full-image dimension and halving for centring is the key technique to show.)

6. Internet of Things (IoT) ★★★

Q: Different components of IoT; how IoT affects everyday life

Main components of an IoT system: 1. **Sensors/Devices:** collect data from the physical environment (temperature, motion, light, etc.). 2. **Connectivity:** the network layer (Wi-Fi, Bluetooth, cellular, LPWAN, etc.) that transmits collected data to processing systems. 3. **Data processing:** software (often cloud-based) that analyzes/processes the incoming raw data. 4. **User interface:** dashboards, mobile apps, or automated actions that present processed information to users or trigger control actions. 5. **(Often added) Actuators:**

devices that act on the physical world based on processed decisions (motors, valves, switches).

Effect on everyday life: IoT enables smart homes (automated lighting, thermostats, security), wearable health monitors, smart cities (traffic and utility management), connected vehicles, and industrial automation — generally increasing convenience, efficiency, real-time monitoring/control, and enabling data-driven decision making in daily activities.

Q: Communication models in IoT

Common models include: 1. **Device-to-Device (D2D):** two or more devices connect and communicate directly (e.g., via Bluetooth) without an intermediary server. 2. **Device-to-Cloud (D2C):** the IoT device communicates directly with a cloud service to send/receive data. 3. **Device-to-Gateway (D2G):** the device connects through a local gateway (which may perform protocol translation or local processing) before reaching the cloud. 4. **Back-End Data-Sharing:** cloud infrastructure allows data collected from IoT devices to be aggregated and shared with third-party services/analysed collectively.

Impact: each model has different implications for **data transmission** (direct vs. relayed), **scalability** (gateway/cloud models scale better to many devices), and **energy efficiency** (D2D can be more efficient for local, short-range tasks, while cloud models require more continuous connectivity/power for long-range transmission).

Q: Internet of Underground Things and Internet of Underwater Things

- **Internet of Underground Things (IoUT):** a network of sensors and communication devices deployed below the earth's surface (soil, mines, pipelines) to monitor conditions such as soil moisture, structural integrity, or seismic activity, using wireless underground communication which must cope with high signal attenuation through soil/rock.
- **Internet of Underwater Things (IoUT - underwater variant):** a network of sensors, autonomous vehicles, and communication nodes deployed underwater (sea, lakes) to monitor aquatic conditions (pollution, marine life, underwater structures), typically relying on acoustic (rather than radio) communication because radio waves attenuate rapidly in water.

Q: Three key dimensions of IoT and how they enable a seamless ecosystem

IoT is often described along three dimensions: 1. **"Things"/Devices dimension:** the physical objects/sensors/actuators being connected. 2. **Intelligence dimension:** the data-processing and analytics layer (often AI/ML-driven) that extracts meaningful insight from raw sensor data. 3. **Connectivity/Network dimension:** the communication infrastructure linking devices to each other and to processing/storage systems.

Together, these three dimensions enable a seamless ecosystem because the devices continuously gather real-world data, the network reliably transports that data, and the intelligence layer turns it into actionable information — closing a feedback loop from sensing to decision to action without manual intervention.

Q: Types of IoT security measures / best practices

- Strong, unique device authentication and credential management (avoiding default passwords).
- Data encryption both in transit and at rest.
- Regular firmware/software updates and patch management.
- Network segmentation (isolating IoT devices from critical infrastructure/networks).
- Use of secure communication protocols (TLS/DTLS) and secure boot mechanisms.
- Continuous monitoring/intrusion detection for anomalous device behavior.
- Physical security of devices to prevent tampering.

Q: Important communication protocols used by IoT

- **MQTT (Message Queuing Telemetry Transport):** a lightweight publish-subscribe protocol ideal for low-bandwidth, high-latency, or unreliable networks; devices publish data to "topics" on a broker, and subscribers receive updates.
- **CoAP (Constrained Application Protocol):** a lightweight request-response protocol (similar to HTTP but simplified) designed for constrained devices and networks.
- **HTTP/HTTPS:** used where devices have enough resources to support standard web protocols.
- **Zigbee/Z-Wave:** low-power, short-range mesh networking protocols for home automation.
- **M2M (Machine-to-Machine):** general term for direct communication between devices without human intervention, often using cellular or dedicated M2M protocols/modules.
- **Bluetooth Low Energy (BLE):** short-range, very low power, used in wearables and personal-area IoT devices.

Q: Key components/characteristics of a digital ecosystem; challenges and opportunities

Key components: interconnected devices/platforms, data (as the core asset flowing between components), software/applications/services, users (producers and consumers of data/services), and the underlying network infrastructure connecting them all, often supported by cloud/analytics platforms.

Characteristics: interdependence between participants (similar to a biological ecosystem), continuous evolution/adaptation, decentralization in many implementations, and value creation through data exchange.

Challenges: interoperability between heterogeneous devices/standards, data privacy and security, scalability as the number of connected entities grows, managing/maintaining a large distributed system.

Opportunities: new business models built on data/services, improved efficiency through automation, innovation through combining data across previously separate domains, and better decision-making through aggregated analytics.

7. Keyboards & Interfacing ★★★

Q: What is key debouncing? Why is it needed?

Mechanical keyboard switches have physical metal contacts that, due to mechanical springiness, do not close cleanly in a single instant — instead the contacts **bounce**, opening and closing rapidly several times over a few milliseconds before settling into a stable closed (or open) state. **Debouncing** is the process (in hardware via an RS latch/capacitor filter, or in software via a short timed delay/re-check) of filtering out these spurious multiple transitions so that a single key press is registered as exactly one logical keystroke.

Why needed: without debouncing, a single physical key press could be interpreted by the processor as several rapid presses (because it samples fast enough to catch the bounce), causing repeated/erroneous characters to be entered.

Q: Define make code, break code, typematic, persistence, aspect ratio, resolution

- **Make code:** the unique code sent by the keyboard controller to the processor when a key is **pressed down**.
- **Break code:** the code sent when a key is **released**, typically the make code with an additional bit/prefix indicating release.
- **Typematic:** the automatic repetition of a character's code when a key is held down continuously for longer than a set delay (auto-repeat feature).
- **Persistence:** in display systems, the time a phosphor dot continues to emit light after the electron beam has excited it (how long the glow "persists" before fading).
- **Aspect ratio:** the ratio of vertical points to horizontal points needed to produce equal-length lines in both directions on a display (or more generally, the ratio of width to height of the screen/image).
- **Resolution:** the number of distinct points (pixels) that can be displayed/distinguished per unit length/area on a screen — higher resolution means more pixels and finer detail.

Q: Four switching methods used when a key is pressed

1. **Mechanical switch:** simple metal contacts physically touch to complete a circuit when the key is pressed.
2. **Membrane switch:** a flexible plastic membrane with printed conductive traces; pressing the key pushes two conductive layers together to complete a circuit.
3. **Capacitive switch:** pressing the key changes the capacitance between two plates under the keycap; the change in capacitance (rather than a direct electrical contact) is detected to register a keypress — no physical contact bounce, very durable.
4. **Hall-effect switch:** a small magnet moves closer to a Hall-effect sensor as the key is pressed; the resulting change in the sensor's output voltage (proportional to magnetic field strength) registers the keypress — completely contactless, very long life.

Q: Software keyboard interfacing flowchart for a 4x4 keyboard, step by step

A 4x4 matrix keyboard has 4 row lines and 4 column lines (16 keys at their intersections), usually interfaced through an I/O port (e.g., via 8255).

Flowchart steps: 1. **Initialize:** configure the row lines as outputs and column lines as inputs (or vice versa) through the I/O port. 2. **Scan for any key press:** output all rows to logic 0 (active low) simultaneously, then read the column lines; if all columns read high (1), no key is pressed — loop back and keep scanning. 3. **Debounce delay:** if any column reads low, introduce a short software delay (a few milliseconds) and re-check the columns to confirm it is a genuine press, not contact bounce. 4. **Row-by-row scan:** if confirmed, activate (drive low) one row at a time (starting from row 0), and after each row activation, read the column lines. 5. **Identify key:** the row that is active *and* the column line that reads low together identify the specific key at that row-column intersection; look up this row/column combination in a table to get the corresponding key code (make code). 6. **Wait for key release:** continue monitoring until all columns return high, confirming the key has been released (and can apply another debounce delay here). 7. **Return the key code** to the calling program / send it to the processor, then loop back to step 2 to scan for the next key press.

Q: Modern keyboards support bidirectional data transfer — how?

Modern keyboards (e.g., PS/2 or USB-based) support bidirectional communication: the keyboard sends scan codes (make/break codes) to the host, while the host can also send commands/data back to the keyboard — for example, commands to turn the Num Lock, Caps Lock, or Scroll Lock LED indicators on/off, set the typematic rate/delay, or reset the keyboard. This is achieved using a clock and data line pair (in PS/2) where either device can initiate a transfer following an agreed protocol (the host has priority and can inhibit the keyboard from sending by pulling the clock line low), or via the USB protocol's standard control/interrupt endpoints that allow host-to-device reports (e.g., HID "Set_Report" for LEDs) alongside device-to-host input reports.

8. 8279 Programmable Keyboard/Display Controller ★★★

Q: Architectural diagram of 8279; how does it work?

The **8279** is a programmable device that simultaneously handles keyboard scanning/debouncing and display refresh, reducing the load on the main processor.

Main internal blocks: 1. **I/O Control and Data Buffer:** interfaces the 8279 to the system data bus (D0–D7) and manages RD, WR, CS, A0 control signals. 2. **Control and Timing Registers, Timing and Control:** contains a programmable clock divider (to generate internal scan timing from the system clock) and the control logic that sequences keyboard scanning and display refresh operations based on the mode set by the command word. 3. **Scan Counter:** generates the scanning sequence for both keyboard row scan lines and display digit selection (in encoded or decoded scan mode), cycling through the rows/digits. 4. **Return Buffers & Keyboard Debounce and Control:** the return lines (RL0–RL7) read the state of the columns during a scan; debounce logic confirms a genuine keypress (not bounce), and 2-key lockout or N-key rollover logic resolves multiple simultaneous key presses. 5. **8x8 FIFO/Sensor RAM and Status logic:** stores key codes (as they are detected) in a FIFO queue so multiple keystrokes aren't lost even if the processor is busy; also generates an interrupt (IRQ) to the processor when data is available. 6. **Display Address Registers and Display RAM:** stores the data to be displayed on each of up to 16 digits; the scan counter cycles through, and this RAM's contents are sent out via the OUTA/OUTB output lines to drive display digits in sync with the scan lines.

Working: the 8279 continuously scans the keyboard matrix (via the scan lines SL0–SL3) in the background, independent of the processor; when a key is detected and debounced, its code is placed in the internal FIFO and an interrupt line (IRQ) is raised to alert the processor to read it. Simultaneously, and independently, it also cycles through display RAM content and drives it out to refresh a multiplexed multi-digit display, without requiring continuous processor intervention for either function.

Q: Operational modes of 8279; how many ways is the keyboard interfaced with the CPU

Operational modes (via the command word): - **Input modes:** Scanned Keyboard mode with 2-Key Lockout, Scanned Keyboard mode with N-Key Rollover, Scanned Sensor Matrix mode, and Strobed Input mode. - **Display modes:** 8-digit or 16-digit display, with left-entry or right-entry data format, and encoded or decoded scan output.

Ways the keyboard can be interfaced with the CPU: 1. **Polled mode:** the CPU periodically checks (polls) the 8279's status register to see if a key has been pressed. 2. **Interrupt mode:** the 8279 raises an interrupt (IRQ) to the CPU whenever a valid keystroke is available in its FIFO, so the CPU only responds when needed rather than continuously polling.

Q: Programmed I/O vs Interrupt-initiated I/O

- **Programmed I/O:** the CPU continuously checks (polls) the status of an I/O device in a loop, waiting until the device signals it is ready, before transferring data — the CPU is fully occupied/busy-waiting during this time, wasting processing time on slow devices.
- **Interrupt-initiated I/O:** the CPU continues executing other tasks and is only interrupted by the device (via an interrupt request line) when the device is actually ready for a data transfer; the CPU then suspends its current task, services the I/O request (transfers data), and resumes what it was doing — far more efficient use of CPU time than polling.

9. Printers ★★★

Q: What is an Impact printer? How does it work? Differentiate impact vs non-impact.

Impact printer: a printer that forms characters/images on paper by physically striking an ink ribbon against the paper (mechanical impact), similar to a typewriter — e.g., dot-matrix printers, daisy-wheel printers, line printers.

Working (dot-matrix example): a print head containing a matrix of small pins strikes an inked ribbon against the paper; the specific pins that fire, in a particular pattern as the head moves across the line, form the shape of each character or graphic dot-by-dot.

Aspect	Impact Printer	Non-Impact Printer
Mechanism	Physical striking of ribbon against paper	No physical contact; uses ink spray, laser/toner, or heat
Noise	Noisy (mechanical impact)	Quiet
Speed	Generally slower	Generally faster
Print quality	Lower resolution, more mechanical wear	High resolution, sharper output
Multi-copy (carbon) printing	Can print carbon-copy forms (pressure-based)	Cannot produce carbon copies
Examples	Dot-matrix, daisy-wheel, line printer	Inkjet, laser printer

Q: How does an inkjet printer work?

An inkjet printer forms images by spraying tiny, precisely controlled droplets of liquid ink directly onto the paper. The print head contains many tiny nozzles; ink is forced through each nozzle either by rapidly heating a small ink chamber to create a vapor bubble that ejects a droplet (thermal/bubble-jet method), or by using a piezoelectric crystal that changes shape when a voltage is applied, mechanically forcing a droplet out (piezoelectric method). The print head moves across the paper line by line, and the pattern of droplets ejected at each position builds up the text/image, with color reproduced by combining cyan, magenta, yellow, and black ink droplets.

Q: How does a laser printer work? Functional diagram

Working steps of a laser printer (electrophotographic process): 1. **Charging:** a photosensitive drum is given a uniform electrostatic charge. 2. **Exposing (writing):** a laser beam, steered by a rotating mirror, scans across the drum and selectively discharges points on it corresponding to the image/text to be printed (the discharged areas represent where toner should stick). 3. **Developing:** the drum rotates past a toner reservoir; oppositely charged toner particles are attracted to the discharged (image) areas of the drum. 4. **Transferring:** the paper, given an opposite charge from underneath, passes close to the drum, pulling the toner pattern off the drum and onto the paper. 5. **Fusing:** the paper then passes through a heated fuser roller assembly, which melts the toner particles and permanently bonds them to the paper. 6. **Cleaning:** any residual toner is removed from the drum, and its charge is neutralized, preparing it for the next print cycle.

Q: Steps involved in starting a printing sequence using a printer controller chip

1. The processor checks the printer's status (via the controller) to ensure it is ready (not busy, has paper, no error condition).
2. The processor sends the data byte(s) to be printed to the controller's data register/buffer.
3. The controller asserts a "strobe" signal to the printer to indicate valid data is present.
4. The printer reads the data, processes/prints the character, and responds with an "acknowledge" (or busy) signal back to the controller.
5. The controller (and processor) waits for the acknowledge/ready signal before sending the next byte, repeating the cycle for subsequent data.

Q: Eight major communication types for a printer interface

1. Parallel (Centronics) interface
2. Serial (RS-232) interface
3. USB interface
4. Network (Ethernet/TCP-IP) interface
5. Wireless (Wi-Fi) interface
6. Bluetooth interface
7. Infrared (IrDA) interface
8. SCSI interface

10. USB & Serial/Parallel Buses ★★★

Q: Define USB; functions of USB ports; limitations of USB

USB (Universal Serial Bus): a standard serial-bus interface for connecting peripherals to a computer, providing both data communication and (for many devices) electrical power over a single cable/connector.

Functions of USB ports: allow plug-and-play connection/detection of devices without rebooting; supply power to low-power peripherals directly from the port; support multiple device classes (storage, HID, audio, video) via a common connector and standardized protocol; allow daisy-chaining/hub expansion of many devices from one host port.

Limitations: cable length is limited (typically a few meters per segment without active extension); total power available per port is limited (insufficient for very high-power devices without external power); bandwidth, while high, is still shared among all devices on the same hub/bus; not ideal for extremely long-distance or highly noise-resistant industrial applications compared to some other buses.

Q: IEEE 1394 (FireWire) vs USB

Aspect	IEEE 1394 (FireWire)	USB
Topology	Peer-to-peer (devices can talk directly without a host)	Host-centric (all transfers go through/controlled by the host)
Speed (historical)	Often faster for sustained high-bandwidth transfer (e.g., video)	Historically variable, improved greatly in later versions (USB 3.x)
Cost	Generally more expensive to implement	Cheaper, more widely adopted
Usage	Popular for professional audio/video equipment	Universal — virtually all consumer peripherals

Q: Three-tier topology of USB; benefits

USB uses a **tiered-star topology**: a **Root hub** (tier 1, built into the host controller) connects to one or more **hubs** (tier 2), which in turn connect to additional hubs or directly to **end devices** (tier 3), forming a tree structure with the host at the root.

Benefits: allows many devices (up to 127) to be connected to a single host controller through hub expansion; simplifies cabling (star topology avoids long bus cabling runs); a fault or unplugging at one branch does not necessarily affect devices on other branches; supports hot-plugging and easy expansion by simply adding another hub.

Q: USB transaction — up to three packet types

1. **Token packet:** sent by the host to initiate a transaction, specifying the transfer type (IN/OUT/SETUP) and target device/endpoint address.
2. **Data packet:** carries the actual payload data being transferred, in either direction (host to device or device to host), following the token.
3. **Handshake packet:** sent to confirm successful reception of data (ACK), indicate the receiver is busy and cannot accept data now (NAK), or report a transfer error (STALL).

Q: How USB peripherals communicate with the microprocessor; USB Type-C 3.0 pinout

USB peripherals communicate with the host/microprocessor through the **host controller** (built into the chipset), which manages the USB protocol stack: the host polls/schedules transactions, the device responds via its endpoints (control, bulk, interrupt, or isochronous), and data ultimately reaches the processor through the host controller's registers/DMA into system memory, from where the operating system's USB driver delivers it to application software.

USB Type-C pinout (basic): the Type-C connector is reversible/symmetric and includes: multiple pairs of **SuperSpeed differential data lines (TX1+/-, RX1+/-, TX2+/-, RX2+/-)** for high-speed data (used in USB 3.x), a pair of **USB 2.0 D+/D-** lines for backward compatibility, **VBUS** and **GND** pins for power, **CC1/CC2 (Configuration Channel)** pins used for cable orientation detection, power-role negotiation and USB Power Delivery signaling, and **SBU1/SBU2 (sideband use)** pins for alternate-mode signaling (e.g., DisplayPort).

Q: What is PCI Express? Features and applications

PCI Express (PCIe): a high-speed serial computer expansion bus standard that replaced the older parallel PCI/AGP buses, using point-to-point serial links ("lanes") instead of a shared parallel bus.

Features: scalable link widths (x1, x4, x8, x16 lanes) allowing different bandwidth levels for different devices; full-duplex communication (simultaneous send/receive); much higher bandwidth per pin than parallel PCI; supports hot-plugging and advanced power management; backward-compatible software model with older PCI.

Applications: graphics cards, solid-state drives (NVMe), network interface cards, and other high-bandwidth expansion cards in modern PCs and servers.

Q: Serial ATA vs Parallel ATA; why SATA is used instead of PATA

Aspect	Parallel ATA (PATA)	Serial ATA (SATA)
Data transmission	Multiple bits sent simultaneously over wide ribbon cable	Bits sent serially, one after another, over thin cable
Cable	Wide 40/80-pin ribbon cable, short max length	Thin cable, longer max length, better airflow
Speed	Lower maximum throughput; also suffers from crosstalk/skew at high speeds	Much higher throughput; less susceptible to timing skew/crosstalk
Hot-plugging	Not supported	Supported
Connector	Bulky, limits devices per cable (master/slave jumpers needed)	Simple point-to-point connection per drive

Why SATA is preferred: the serial approach avoids the timing-skew and crosstalk problems that limit how fast parallel signals can be reliably sent together, allowing much higher data rates over a thinner, longer, more flexible cable, while also enabling hot-swapping and simpler cabling/airflow inside the case.

Q: ISA vs VESA vs EISA / VESA vs PCI

- **ISA (Industry Standard Architecture):** original 8/16-bit expansion bus, low speed, no auto-configuration.
- **EISA (Extended ISA):** a 32-bit extension of ISA, backward compatible, higher bandwidth, but still relatively slow and largely superseded.
- **VESA (VESA Local Bus, VLB):** connected expansion cards more directly to the CPU's local bus for higher-speed graphics performance, but was processor-speed-dependent and less flexible/scalable, eventually replaced by PCI, which is processor-independent, supports plug-and-play, and offers more consistent performance across different CPU types/speeds.

Q: Short notes — AGP, Plug and Play, SCSI

- **AGP (Accelerated Graphics Port):** a dedicated high-speed expansion slot/bus specifically designed for graphics cards, providing a direct, fast point-to-point connection to the system's memory controller for improved 3D graphics performance (now superseded by PCIe).
- **Plug and Play (PnP):** a standard allowing the operating system to automatically detect and configure newly connected hardware devices (assigning resources like IRQs, I/O addresses) without manual user configuration.
- **SCSI (Small Computer System Interface):** a set of standards for physically connecting and transferring data between computers and peripheral devices (hard drives, scanners, tape drives), supporting multiple devices on a single bus/chain with each device having a unique ID, historically offering higher performance and multiple-device support versus IDE.

Q: IRQ vs DMA channels

- **IRQ (Interrupt Request):** a signal line a device uses to interrupt the CPU, requesting immediate attention/service (e.g., to indicate data is ready); the CPU's normal execution is paused to run an interrupt service routine.
- **DMA (Direct Memory Access) channel:** a dedicated channel allowing a peripheral device to transfer data directly to/from main memory **without** continuous CPU involvement for each byte, freeing the CPU to perform other tasks while a bulk data transfer happens in the background (the CPU is only involved at the start/end of the transfer).
- **Key difference:** IRQ is about getting the CPU's attention for a task/event; DMA is about moving bulk data efficiently while bypassing the CPU for the actual transfer itself.

11. Number Systems & Floating Point ★★

Q: Convert a decimal number to IEEE 32-bit floating point format

Method (shown for 39887.5625, apply the same method for any given number, e.g., -1313.3125): 1. **Sign bit:** 0 for positive, 1 for negative. (39887.5625 is positive, so sign = 0.) 2. **Convert to binary:** 39887 in binary = 1001101111001111; 0.5625 in binary = 0.1001 (0.5625 = $1/2 + 1/16 = 0.5 + 0.0625$). So 39887.5625 = 1001101111001111.1001 in binary. 3. **Normalize:** write as $1.xxxxx \times 2^n$ by moving the binary point so only one '1' remains before it: 1001101111001111.1001 = $1.0011011110011111001 \times 2^{15}$ (shifted 15 places, since there are 16 integer bits, so the point moves left by 15 to leave one leading 1). 4. **Exponent (biased):** IEEE-754 single precision uses a bias of 127. Biased exponent = actual exponent + 127 = 15 + 127 = 142. In binary (8 bits): 142 = 10001110. 5. **Mantissa (fraction):** take the 23 bits after the leading 1 (which is implied/hidden and not stored): 00110111100111110010000 (pad or truncate to exactly 23 bits). 6. **Assemble the 32-bit word:** Sign (1 bit) | Exponent (8 bits) | Mantissa (23 bits) = 0 | 10001110 | 00110111100111110010000. For a **negative** number (e.g., -1313.3125), follow the identical steps but set the sign bit to 1; convert the magnitude (1313.3125) to binary, normalize, bias the exponent, and take 23 mantissa bits exactly as above.

Q: Interpret a hex value (e.g., EFE1H) as unsigned, signed, and floating point

- **As unsigned integer:** simply convert the hex value directly to its decimal equivalent (e.g., EFE1H = 61409 in decimal), treating all 16 bits as magnitude.
- **As signed integer (2's complement, 16-bit):** check the MSB (top bit of E = 1110, so MSB=1, meaning negative). To find the value: take 2's complement of EFE1H — invert bits and add 1: EFE1H → invert = 101E H... (perform digit-wise binary inversion then +1) to get the magnitude, then report it as a negative decimal number (i.e., the value equals $-(65536 - 61409) = -4127$).
- **As a floating-point interpretation:** if the 16 (or extended to 32) bits are instead read according to an IEEE-754-style layout (sign bit, exponent bits, mantissa bits), decode the top bit as sign, the next group of bits as a biased exponent, and the remaining bits as the mantissa/fraction, then reconstruct the value as $(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{(\text{exponent}-\text{bias})}$, exactly reversing the encoding procedure shown in the previous question.

Q: How are signed binary numbers represented? Range of 20-bit signed values

Signed binary numbers are most commonly represented using **2's complement**: the MSB indicates sign (0 = positive, 1 = negative), and negative values are formed by inverting all bits of the positive magnitude and adding 1. (Other methods exist: sign-magnitude, and 1's complement, but 2's complement is standard in modern processors because it has a single representation for zero and simplifies arithmetic circuits.)

Range for n = 20 bits (2's complement): from $-2^{(n-1)}$ to $+2^{(n-1)} - 1 = -2^{19}$ to $+2^{19} - 1 = -524288$ to $+524287$.

Q: Perform binary addition of two signed decimal numbers; determine carry, sign, overflow, zero flags

General method: 1. Convert both decimal numbers to their n-bit 2's complement binary representation. 2. Add the two binary numbers using standard binary addition rules. 3. **Carry flag (CY):** set to 1 if there is a carry-out of the most significant bit. 4. **Sign flag (S):** equal to the MSB of the result (1 = result is negative, 0 = positive). 5. **Overflow flag (OF):** set to 1 if the carry INTO the MSB differs from the carry OUT of the MSB (i.e., if you add two numbers of the same sign and get a result of the opposite sign, overflow has occurred). 6. **Zero flag (Z):** set to 1 if the result is all-zero bits. (Apply this procedure directly to whichever specific pair of numbers the

exam gives — compute the actual binary sum, then check each flag using the rules above.)

Q: When is an overflow flag set or reset? Examples

The **overflow flag** is set (1) when an arithmetic operation on two **signed** numbers produces a result that does not fit within the representable range for the given number of bits — specifically, when adding **two positive numbers yields a negative result**, or **two negative numbers yield a positive result** (the carry into the sign bit differs from the carry out of the sign bit). It is reset (0) otherwise, including in cases where a carry occurs but does not cause a sign error (e.g., adding a positive and negative number never causes overflow).

Examples (8-bit, 2's complement): - 01111111 (+127) + 00000001 (+1) = 10000000, which reads as -128 — a positive + positive giving a negative result = **overflow set**. - 10000000 (-128) + 11111111 (-1) = 01111111, which reads as +127 — negative + negative giving a positive result = **overflow set**. - 00000010 (+2) + 00000011 (+3) = 00000101 (+5), a correct result within range = **overflow reset**.

12. Serial Communication / RS-232 / 8251A ★★

Q: Functional block diagram of 8251A PCI/USART; explain each section

The **8251A** is a Universal Synchronous/Asynchronous Receiver/Transmitter (USART), converting parallel data from the microprocessor into serial form for transmission, and vice versa.

Main functional sections: 1. **Data Bus Buffer:** an 8-bit bidirectional buffer connecting the 8251A to the system data bus, transferring control words, status information, and data between the CPU and the chip. 2. **Read/Write Control Logic:** accepts control signals (RD, WR, CS, C/D — command/data select) from the CPU to determine whether the CPU is reading status, writing a control/command word, or transferring actual data. 3. **Transmitter section (Transmit Buffer & Transmit Control):** accepts parallel data from the CPU, adds necessary framing bits (start bit, optional parity, stop bit(s)) according to the programmed format, and shifts the data out serially on the TxD line, synchronized to the transmit clock (TxClk). 4. **Receiver section (Receive Buffer & Receive Control):** accepts serial data on the RxD line, strips off the start/stop/parity bits, checks for framing/parity errors, and assembles the data back into parallel form for the CPU to read, synchronized to the receive clock (RxClk). 5. **MODEM Control:** manages handshaking signals such as RTS (Request to Send), CTS (Clear to Send), DSR (Data Set Ready), and DTR (Data Terminal Ready) for interfacing with modems or other serial devices requiring handshaking.

Q: RS-232 Handshaking / asynchronous communication protocol; RS-232C connection example

RS-232 is a standard for serial, asynchronous data communication between a DTE (Data Terminal Equipment, e.g., computer) and DCE (Data Communication Equipment, e.g., modem), defining voltage levels, connector pinouts, and control-signal handshaking.

Asynchronous communication: data is sent one character at a time, with each character framed independently by its own **start bit** (signals the beginning of a character, always a “space”/0), the **data bits**, an optional **parity bit** (for basic error checking), and one or more **stop bit(s)** (marks the end of the character, always a “mark”/1) — no separate clock line is shared; the receiver resynchronizes with each start bit.

Handshaking signals: **RTS (Request To Send)** raised by the DTE to ask if it may transmit; **CTS (Clear To Send)** raised by the DCE in response, confirming it is ready to receive; **DTR (Data Terminal Ready)** and **DSR (Data Set Ready)** confirm that each end of the link is powered on and ready to operate.

Simple RS-232C connection example: two devices connected via a cable using pins for TxD (transmit data), RxD (receive data), and a common signal ground (GND) at minimum, plus optionally RTS/CTS and DTR/DSR for full hardware handshaking, over a defined voltage range (traditionally ± 3 to ± 15 V, with negative voltage = logic 1/mark and positive voltage = logic 0/space).

Q: ASCII 'A' framed transmission, character-oriented transmission

For a character-oriented (asynchronous) transmission of ASCII 'A' (0100 0001) framed between a start bit and two stop bits with LSB sent first: the line is normally held at the “mark” (idle, logic 1) state; transmission begins with a **start bit** (logic 0) signalling the receiver to begin sampling; then the **7 or 8 data bits of 'A' are sent LSB-first** (i.e., bit 0 first, bit 7/MSB last); then (optionally) a parity bit; then **two stop bits** (logic 1) mark the end of the character and return the line to idle, allowing the receiver time to prepare for the next character.

Bits per character = start bit(s) + data bits + parity (if used) + stop bits (here: 1 + 8 + 0 + 2 = 11 bits total for this example). Data begins right after the start bit is detected and ends after the last data (or parity) bit, with the stop bit(s) simply providing an idle gap before the next start bit can appear. **Synchronous vs asynchronous:** synchronous transmission sends a continuous stream of bits with a shared clock (no per-character start/stop overhead, higher efficiency for large data blocks) while asynchronous transmission (as described) frames each character independently with start/stop bits, which is simpler and tolerant of timing drift, but has more overhead per character — for short, irregular/interactive data (like keyboard input), asynchronous is generally the better/typical method, whereas synchronous is preferred for high-throughput bulk data transfer.

Q: DMA-controlled communication process; block diagram of a typical DMA controller

Data transfer between memory and I/O controlled by an **external circuit** (rather than the CPU executing instructions for each byte) is called **DMA (Direct Memory Access)**.

Basic block diagram elements of a DMA controller: - **Address register(s):** hold the current memory address being read from/written to. - **Word count register:** holds the number of words/bytes remaining to be transferred, decremented after each transfer. - **Control logic/Control register:** determines the transfer direction (read/write), transfer mode (single, block, demand), and generates the necessary control signals (MEMR/MEMW, IOR/IOW). - **Bus request/Bus grant lines (HRQ/HLDA):** used by the DMA controller to request control of the system bus from the CPU, and by the CPU to grant it, temporarily disconnecting itself from the bus during the transfer (the CPU enters a HOLD state). Once granted control, the DMA controller directly drives the address and control lines to move data between the I/O device and memory without CPU instruction execution for each byte, notifying the CPU (often via interrupt) once the word count reaches zero (transfer complete).

Q: Basic principles of the two-wire handshaking method of data transfer

Two-wire handshaking uses two control signals (typically named “Data Valid/STB” from the sender and “Data Accepted/ACK” from the receiver) to synchronize a transfer without a shared clock: 1. The sender places data on the data lines and asserts the “Data Valid” signal to indicate the data is ready. 2. The receiver, upon detecting “Data Valid,” reads the data and then asserts “Data Accepted” to acknowledge receipt. 3. The sender, upon seeing “Data Accepted,” removes/de-asserts “Data Valid” (and the data), signalling the receiver may reset its “Data Accepted” line. 4. Once both lines return to their idle states, the cycle can repeat for the next data item. This full cycle of mutual signalling ensures neither side proceeds faster than the other can handle, providing reliable asynchronous data transfer.

Q: IEEE 488 bus system; operation and pin configuration

The **IEEE-488 bus (GPIB — General Purpose Interface Bus)** is a parallel bus standard originally developed for connecting laboratory/test instruments to computers, supporting multiple devices (up to 15) on a single bus.

Operation based on handshaking: IEEE-488 uses a **three-wire handshake** with the lines **DAV (Data Valid)**, **NRFD (Not Ready For Data)**, and **NDAC (Not Data Accepted)**: the talker (sending device) waits until all listeners indicate they are ready (NRFD goes false/high for all), then places data on the 8 data lines and asserts DAV; each listener then individually indicates it has accepted the data by releasing NDAC, and only once **all** listeners have accepted does DAV get released, allowing the next byte to be sent — ensuring the transfer proceeds only as fast as the *slowest* listening device can handle it.

Pin configuration (basic): 8 data lines (DIO1–DIO8), 3 handshake lines (DAV, NRFD, NDAC), and 5 general interface management lines (ATN – Attention, IFC – Interface Clear, SRQ – Service Request, REN – Remote Enable, EOI – End Or Identify), plus ground lines.

13. Coding & Interrupts ★★

Q: Basic differences between ASCII and EBCDIC

Aspect	ASCII	EBCDIC
Full form	American Standard Code for Information Interchange	Extended Binary Coded Decimal Interchange Code
Bit length	Originally 7-bit (128 codes), extended to 8-bit	8-bit (256 codes)
Developer/usage	Widely used in PCs, most modern systems and communications	Developed by IBM, mainly used in IBM mainframe systems
Character ordering	Letters and digits are contiguous and predictably ordered, simplifying sorting	Non-contiguous ordering of characters in some ranges, more complex sorting logic needed

Q: Define software interrupt with example

A **software interrupt** is an interrupt deliberately triggered by executing a special instruction within a program (rather than by an external hardware signal), typically used to invoke operating system services or debugging functions. **Example:** the INT instruction in x86 assembly (e.g., INT 21H in DOS to invoke a BIOS/DOS service routine), or the TRAP/SVC instructions used to make system calls in various processors — executing this instruction causes the processor to jump to a predefined interrupt service routine just as a hardware interrupt would, even though no external device signalled it.

Q: Main types of interrupts

1. **Hardware interrupts:** generated by external devices/peripherals asserting an interrupt request line (e.g., keyboard, timer, I/O completion).
2. **Software interrupts:** generated intentionally by program instructions (as above) to invoke system routines.
3. **Internal (exception) interrupts / traps:** generated automatically by the processor itself due to an internal error condition or

special event during instruction execution (e.g., divide-by-zero error, illegal opcode, overflow trap) — not caused by an explicit external signal or instruction, but by the CPU detecting a problem while executing normally.

Q: Maskable vs Nonmaskable interrupts; handling power failure interrupt

- **Maskable interrupts:** can be selectively enabled or disabled (masked) by the programmer/CPU using an interrupt mask/enable flag; if masked, the CPU will ignore the interrupt request even if it occurs, useful for lower-priority interrupts that can be deferred.
- **Nonmaskable interrupts (NMI):** cannot be disabled/ignored by software under any circumstances; they are reserved for critical events that must always be serviced immediately, such as **power failure detection** or hardware fault conditions.

Handling a power failure interrupt: power failure is typically routed to the CPU's NMI input because it demands guaranteed, immediate attention. On detecting imminent power loss (usually via a backup capacitor providing a few milliseconds of residual power), the interrupt service routine quickly saves the current CPU state (registers, program counter) and any critical volatile data to non-volatile storage before the power is fully lost, so that processing can resume correctly once power is restored.

Q: Three types of I/O; microprocessor-initiated vs device-initiated

1. **Programmed I/O** — microprocessor-initiated: the CPU actively polls/checks the device's status and controls the entire transfer through executed instructions.
2. **Interrupt-initiated I/O** — device-initiated: the device itself signals the CPU (via an interrupt) when it is ready, after which the CPU services the request.
3. **DMA (Direct Memory Access)** — device-initiated (via the DMA controller acting on the device's behalf): data moves directly between the device and memory with minimal CPU involvement, only initiated/terminated by the CPU but the actual bulk transfer is managed autonomously.

Q: Process of a device-initiated I/O transfer

In a device-initiated (interrupt-driven) transfer: the peripheral device, once ready to send or receive data, asserts its interrupt request line; the CPU (if interrupts are enabled and this interrupt isn't masked) completes its current instruction, saves its current state (program counter and relevant registers) onto the stack, and jumps to the corresponding Interrupt Service Routine (ISR) associated with that device; the ISR performs the actual data transfer (reading/writing the device's data register); once the transfer is complete, the ISR restores the CPU's saved state and control returns to the point where the main program was interrupted, resuming normal execution.

14. Floppy Disk Controller & Encoding ★★

Q: Describe Data Read and Data Write operations through FDC

Data Write operation: 1. The CPU issues a Write command to the Floppy Disk Controller (FDC), along with parameters (track, sector, head, number of sectors). 2. The FDC positions the read/write head over the correct track (seek operation) and waits for the correct sector to rotate under the head (using ID field/sector header comparison). 3. Data is transferred from memory to the FDC (often via DMA) and the FDC encodes it (FM/MFM/RLL) before writing it serially to the disk surface through the write head, including generating and appending the necessary sync bytes, address marks, and CRC (cyclic redundancy check) for error detection.

Data Read operation: 1. The CPU issues a Read command with the target track/sector/head parameters. 2. The FDC seeks to the correct track and searches for the matching sector ID field as it rotates past the head. 3. Once found, the FDC reads the serial bit stream from the disk, decodes it (reversing the FM/MFM/RLL encoding) back into parallel data bytes, checks the CRC for errors, and transfers the resulting data to memory (often via DMA) for the CPU to use.

Q: Encode data 101000100110 using FM, MFM, and RLL — show waveforms

FM (Frequency Modulation) encoding: a clock pulse is inserted before every single data bit, regardless of its value; a data '1' is represented by an additional transition in the middle of the bit cell, while a data '0' has no mid-cell transition (only the clock transition at the cell boundary). This makes FM simple but low-density (roughly 1 flux transition per bit for worst case).

MFM (Modified FM) encoding: removes unnecessary clock transitions — a clock pulse is only inserted between two consecutive '0' data bits (when there wasn't already a transition from a preceding '1'); a data '1' still gets a mid-cell transition, but clock transitions between two '1's, or between a '1' and following '0', are omitted since they aren't needed for synchronization. This roughly doubles the recording density compared to FM for the same physical media.

RLL (Run Length Limited) encoding: uses a more complex substitution scheme that further limits the *maximum* and *minimum* number of consecutive zero-bits (no-transition intervals) allowed, translating the actual data into a longer, specially chosen bit pattern via a lookup table designed to guarantee timing recoverability while maximizing bit density — this pushes storage density even higher than MFM (commonly 2–3x FM's density) at the cost of more complex encode/decode circuitry.

For the given data bit stream **1 0 1 0 0 0 1 0 0 1 1 0**, sketch the waveform for each scheme by applying the above rules bit-by-bit: for **FM**, mark a clock transition at every bit-cell boundary, and add a mid-cell transition only for bits that are '1' (bits 1, 3, 7, 10, 11 in this stream); for **MFM**, keep the mid-cell transition for every '1' bit, and add a boundary transition only between two consecutive '0's (check each 0-bit in the sequence and see if its predecessor was also 0 — if so, insert a boundary transition, otherwise leave it flat); RLL waveforms are drawn using the specific code-table substitution your course prescribes (commonly RLL 2,7), which should be applied bit-group by bit-group rather than bit-by-bit like FM/MFM. (Draw three separate timing diagrams, one row of 12 bit-cells each, with vertical tick marks showing where transitions occur according to the rules above — this is the key diagram the exam is grading.)

Q: What is 'CHRN' value? IDE command execution protocol

CHRN stands for **Cylinder, Head, Record (Sector), Number (of bytes)** — the four parameters used by a Floppy Disk Controller to uniquely address a specific sector on the disk: **C** (which cylinder/track), **H** (which read/write head, i.e., which disk surface), **R** (which sector/record number on that track), and **N** (a code indicating the number of bytes per sector, e.g., N=2 typically meaning 512 bytes/sector). Every read/write command sent to the FDC specifies these four values to precisely locate the desired data.

IDE command execution protocol (brief): the host writes the target CHS (cylinder-head-sector) or LBA address and the command code (read/write/identify, etc.) into the drive's task-file registers; the host then checks the drive's status register (waiting for the BUSY flag to clear and DRDY/DRQ to be set as appropriate); once the drive signals it is ready, data is transferred a sector at a time between the host and the drive's data register (via PIO or DMA); the drive raises an interrupt (or the host polls status) to indicate command completion, and the host checks the status/error register for any fault before proceeding to the next command.

15. Human Computer Interaction (HCI) ★★

Q: What is Usability in HCI? Why is it important? Example.

Usability refers to how easily, efficiently, and satisfactorily a user can interact with a system/interface to achieve their intended goals. It is typically evaluated in terms of: **learnability** (how easily new users can accomplish basic tasks), **efficiency** (how quickly experienced users can perform tasks), **memorability** (how easily users can re-establish proficiency after a period of not using it), **error rate/recovery** (how many errors users make and how easily they can recover), and **satisfaction** (how pleasant the interface is to use).

Why important: poor usability leads to user frustration, increased errors, wasted time, lower adoption, and higher training/support costs; good usability increases productivity, user satisfaction, and reduces the learning curve for new users.

Example: a well-designed mobile banking app that lets a first-time user transfer money in a few intuitive taps (high usability) versus a poorly designed one that requires navigating many confusing, unlabelled menus and produces frequent errors (low usability).

Q: Norman's Seven Principles (Stages of Action) for HCI/IoT design

Don Norman's seven stages/principles of interaction design are generally listed as: 1. **Establish the goal**. 2. **Form the intention** (decide what action to take to achieve the goal). 3. **Specify the action sequence** (plan the specific steps). 4. **Execute the action**. 5. **Perceive the state of the world** (observe the result of the action). 6. **Interpret the state of the world** (make sense of what happened). 7. **Evaluate the outcome** (compare the result against the original goal/intention).

These stages are grouped into two "gulfs": the **Gulf of Execution** (the gap between a user's intentions and the actions the system allows) and the **Gulf of Evaluation** (the gap between the system's displayed state and the user's ability to interpret it) — good interface design aims to minimize both gulfs, making it easy for users to figure out what to do and to understand what happened as a result.

Q: What is Human Computer Interaction? Important components of HCI

HCI (Human-Computer Interaction) is the multidisciplinary field of study concerned with the design, evaluation, and implementation of interactive computing systems for human use, and with the study of the major phenomena surrounding them — essentially, how people interact with computers/technology and how to make that interaction more effective, efficient, and satisfying.

Important components: 1. **The user** (human): their cognitive abilities, physical characteristics, and goals/needs. 2. **The computer/system:** the hardware and software that the user interacts with. 3. **The interaction/interface:** the means by which the user and system communicate (input devices, displays, dialogue design, feedback mechanisms). 4. **The task/context:** the specific activity/environment in which the interaction takes place, which shapes what "good" design looks like.

16. Storage Interfaces: RAID, SCSI, ATA ★★

Q: Define RAID; describe different RAID levels

RAID (Redundant Array of Independent/Inexpensive Disks) is a data storage technology that combines multiple physical disk drives into one or more logical units, for purposes of data redundancy, performance improvement, or both.

Common RAID levels: - **RAID 0 (Striping):** data is split (striped) across two or more disks with no redundancy; improves read/write speed significantly, but a single disk failure causes total data loss. - **RAID 1 (Mirroring):** data is duplicated identically across two (or more) disks; provides full redundancy (survives one disk failure) but effectively halves usable capacity. - **RAID 5 (Striping with distributed parity):** data and parity information are striped across three or more disks; can survive a single disk failure (data reconstructed from parity), offering a good balance of redundancy, capacity efficiency, and performance. - **RAID 10 (1+0):** a combination of mirroring and striping — pairs of mirrored disks are then striped together, offering both good performance and good redundancy at the cost of higher hardware requirements.

Q: IDE/ATA vs SCSI; advantages of SCSI over ATA

Aspect	IDE/ATA	SCSI
Devices per bus/cable	Typically 2 (master/slave)	Up to 7 or 15 devices per bus, depending on standard
Performance under multitasking	Lower, since only one transfer at a time is efficient	Better, supports overlapped/multiple concurrent commands
Cost	Cheaper, common in consumer PCs	More expensive, common in servers/workstations
Cable length	Shorter	Longer cable runs supported
CPU overhead	Lower-level, historically more CPU-dependent for control	More intelligent controller, less CPU overhead

Advantages of SCSI over ATA: supports more devices per bus/channel, better performance in multi-device/multitasking environments (can queue and reorder commands), longer cable lengths, and generally higher reliability and hot-swap support suited to server environments.

Q: ATA RAID vs SCSI RAID

Both implement the same RAID concepts (striping/mirroring/parity), but **ATA RAID** arrays are built from (cheaper) ATA/SATA drives, generally offering lower cost but comparatively lower performance and fewer devices per controller, making them common in desktop/small-office use; **SCSI RAID** arrays use SCSI drives with their higher per-bus device count, command queuing, and typically higher reliability/performance, making them the traditional choice for enterprise servers, though at significantly higher cost.

Q: Solid state storage device — meaning, features, advantages

A **solid-state storage device (SSD)** stores data electronically in non-volatile flash memory chips, with no moving mechanical parts (unlike a traditional hard disk drive's spinning platters and moving read/write heads).

Features: uses NAND flash memory cells organized into pages/blocks; includes a controller that manages wear-levelling, error correction, and data mapping.

Advantages: much faster read/write speeds and access times (no mechanical seek delay); more resistant to physical shock/vibration (no moving parts to be damaged); lower power consumption and heat generation; silent operation; typically lighter and more compact than HDDs of equivalent capacity.

17. Buffers, Spoolers & Interfaces ★★

Q: What is a buffer? Why do we use buffers? Differentiate buffers and spoolers.

Buffer: a temporary storage area (in memory or a dedicated chip) used to hold data while it is being transferred between two devices or processes that operate at different speeds, or at different times.

Why used: to compensate for the speed mismatch between a fast device (e.g., CPU/memory) and a slower device (e.g., printer, disk), preventing the faster device from having to wait idly for the slower one on every single data item, and smoothing out irregular bursts of data into a steadier flow.

Buffers vs Spoolers: | Aspect | Buffer | Spooler | |—|—| | Storage location | Usually in main memory (RAM), often small | Usually on disk, can hold much larger amounts of data | | Purpose | Temporary holding to smooth speed mismatch between two immediate operations | Queues entire jobs (e.g., print jobs) for later, potentially out-of-order/asynchronous processing | | Scope | Handles data at a lower, more immediate transfer level | Manages whole jobs/tasks, potentially allowing multiple jobs to queue while a device is busy | | Lifetime | Data typically only exists transiently during the transfer | Job data may persist for a longer period until the resource (e.g., printer) becomes free |

Q: Define interfaces; characteristics of computer peripherals

Interface: the hardware and/or software boundary/mechanism that enables communication and data exchange between two systems or components that may otherwise be incompatible in terms of signal levels, data format, or timing (e.g., between a computer and a peripheral device).

Characteristics of computer peripherals: they are external (or externally-facing) devices that provide input to, or receive output from, the central computer system; they usually operate at different speeds than the CPU (generally much slower); they often require specific interface hardware/protocols (parallel, serial, USB) and device drivers to communicate correctly with the system; they can be classified as input devices (keyboard, mouse, scanner), output devices (monitor, printer), or storage devices (disk drives).

Q: Main functions of an interface

- Signal/voltage level conversion between devices with different electrical characteristics.
- Data format conversion (e.g., parallel to serial or vice versa).
- Speed matching/synchronization (buffering) between devices operating at different rates.
- Providing control/status signalling (handshaking) so both connected devices know the state of the transfer.
- Error detection (and sometimes correction) during data transfer.

18. Wireless Sensor Networks, Real-Time Systems & Misc. ★

Q: Design features essential in WSNs to optimize data collection, communication, and energy efficiency

- **Energy-efficient MAC protocols** (e.g., duty-cycling schemes like S-MAC/T-MAC) to let nodes sleep when not transmitting/receiving, since battery life is the primary constraint in most WSN deployments.
- **Data aggregation/fusion** at intermediate nodes to reduce the total amount of redundant data transmitted across the network, saving both bandwidth and energy.
- **Multi-hop routing protocols** that balance energy consumption across nodes (avoiding always routing through the same “hot spot” nodes, which would deplete their batteries first).
- **Scalability and self-organization**, since WSNs often consist of large numbers of low-cost nodes that must configure themselves and adapt to node failures or topology changes automatically.
- **Low-power hardware design** (efficient radios, low-power sensing circuits, and sleep/wake scheduling at the hardware level).

Q: Primary objective of T-MAC; how it differs from S-MAC in energy efficiency

T-MAC (Timeout MAC) aims to further reduce energy waste (idle listening) compared to earlier protocols by using an **adaptive, variable-length active/listen period** rather than a fixed one: a node stays awake (listening) only until no activity (no incoming traffic, no need to transmit) is observed for a certain timeout interval, after which it immediately goes back to sleep, rather than waiting out a fixed sleep/listen cycle length regardless of actual traffic.

Difference from S-MAC: S-MAC uses a fixed duty cycle (fixed active/sleep period lengths) for all nodes, which can waste energy during idle listening when traffic is low, or cause latency when traffic is bursty; T-MAC dynamically shortens the active period when there is no traffic, adapting to variable/bursty traffic loads and thereby achieving better energy efficiency than S-MAC’s fixed schedule under variable network load, at the potential cost of slightly increased latency or an “early sleeping” problem in certain traffic patterns.

Q: Hidden Terminal Problem and Exposed Terminal Problem in CSMA (WSN MAC)

- **Hidden Terminal Problem:** occurs when two nodes (A and C) are both within range of a common receiver (B) but are **out of range of each other**; since A and C cannot sense each other’s transmissions (carrier sensing fails between them), both may transmit to B simultaneously, believing the channel is free, causing a collision at B that neither sender is aware of.
- **Exposed Terminal Problem:** occurs when a node (B) is transmitting to A, and a nearby node (C) wants to transmit to a *different* node (D) that is out of range of B; C senses the channel busy (because it can hear B’s transmission) and needlessly defers its own transmission, even though its transmission to D would not actually have interfered with A’s reception — wasting available channel capacity unnecessarily. (A diagram showing four nodes in a line, A—B—C—D, with overlapping/non-overlapping range circles, is the standard way to illustrate both cases.)

Q: How does Slotted ALOHA improve upon basic ALOHA? Frame transition diagrams

In **pure (basic) ALOHA**, a node transmits a frame at any arbitrary time as soon as it has data ready, without regard for what other nodes are doing; this maximizes the “vulnerable period” for collisions (a collision can occur if any other node transmits within one full frame-time before or after your transmission starts, giving a vulnerable period of $2 \times \text{frame-time}$), giving a maximum theoretical channel utilization/throughput of about 18%.

Slotted ALOHA improves on this by dividing time into discrete, synchronized **slots** equal to one frame transmission time, and requiring all nodes to begin transmission only at the start of a slot boundary (synchronized via a common clock/beacon signal). This halves the vulnerable period (to just $1 \times \text{frame-time}$, since a node can now only collide with another transmission that started in the *same* slot, not a slightly offset one), roughly doubling the maximum theoretical throughput to about 37%. (A frame-transition diagram shows: for pure ALOHA, overlapping frames starting at arbitrary offsets causing partial-overlap collisions; for slotted ALOHA, frames aligned to fixed slot boundaries, so overlaps only occur when two nodes choose the exact same slot.)

Q: Periodic and aperiodic task models in real-time systems

- **Periodic tasks:** tasks that must execute repeatedly at fixed, regular time intervals (a fixed “period”), e.g., a sensor sampling routine that must run every 10ms; each instance typically has a deadline related to its period.
- **Aperiodic tasks:** tasks that are triggered by irregular, unpredictable events (not on a fixed schedule), e.g., handling an emergency shutdown signal or a user-initiated request; their arrival times are not known in advance, though they may still carry a deadline once triggered. **Example:** in an aircraft control system, continuously reading altitude sensors every fixed interval is a periodic task, while responding to a sudden pilot emergency-override button press is an aperiodic task.

Q: Difference between Embedded and Real-Time Systems

- **Embedded system:** a computer system built into a larger device to perform a specific, dedicated function (e.g., a microwave oven’s controller, a car’s engine control unit); it may or may not have strict timing requirements.
- **Real-time system:** a system whose *correctness* depends not only on producing the right logical result but also on producing it **within a specified time deadline**; missing a deadline is considered a failure (in hard real-time systems) or a degraded/undesirable outcome (in soft real-time systems). **Relationship:** many embedded systems are also real-time systems (e.g., an airbag controller must respond within milliseconds), but not all embedded systems have strict real-time deadlines (e.g., a simple embedded digital clock display), and not all real-time systems are embedded (e.g., a real-time stock-trading server running on general-purpose hardware).

Q: Real-time system for a smart city — incorporating periodic/aperiodic task models

A smart-city real-time system would use **periodic tasks** for continuous, scheduled monitoring activities such as regularly polling traffic-flow sensors, air-quality sensors, or streetlight status at fixed intervals to maintain an up-to-date picture of city conditions, and **aperiodic tasks** for handling irregular, event-driven situations such as responding to an emergency vehicle’s priority signal request at an intersection, a reported accident, or a sudden spike in pedestrian crossing requests — the system’s scheduler would need to guarantee the periodic monitoring tasks meet their deadlines while still reserving enough capacity/priority to service urgent aperiodic events (e.g., using a priority-based or slack-time scheduling algorithm) without indefinitely starving either task type.

Q: Define proximity switches; contact-type configurations; Reed switch applications

Proximity switch: a sensor/switch that detects the presence or absence of a nearby object without requiring direct physical contact (in the non-contact sense) or with only very light contact (in some contact-type variants), typically used to signal a control system when an object comes within a certain range.

Contact-type proximity switch configurations (used in manufacturing automation): simple mechanical **limit switches** mounted so that a passing object physically depresses a lever/plunger, closing (or opening) an electrical contact; **micro-switches** actuated by a cam or moving part on a conveyor/assembly line; and **reed switches**, where a passing magnet (attached to the moving part) closes a pair of ferromagnetic reed contacts sealed inside a glass tube, without requiring the magnet to physically touch the switch housing.

Two applications of a Reed switch: (1) door/window position sensing in security alarm systems (magnet on the door, reed switch on the frame — the circuit opens when the door is opened, triggering the alarm); (2) speed/position sensing on rotating shafts or conveyor systems, where a magnet mounted on the rotating part triggers the reed switch once per revolution/pass, allowing speed or position to be counted.

Q: Characteristics of Real-Time Systems

- **Deterministic timing:** the system must guarantee that tasks complete within their required deadlines, not just complete correctly.
- **Predictability:** system behaviour (worst-case execution time, response time) must be analyzable/predictable in advance.
- **Reliability:** especially in hard real-time systems (e.g., medical devices, flight control), failure to meet a deadline can have severe consequences, so high reliability is essential.
- **Concurrency:** typically must handle multiple simultaneous tasks/events (sensors, actuators, communication) and schedule them appropriately.
- **Classification:** systems are generally classified as **hard real-time** (missing any deadline is a total system failure, e.g., airbag deployment) or **soft real-time** (occasional missed deadlines degrade quality but don’t cause total failure, e.g., video streaming).

Q: Common scanner problems and troubleshooting tips

Common problems: poor/blurry scan quality (dirty scanner glass or outdated drivers), scanner not detected by the computer (loose/faulty USB cable or driver issue), streaks or lines in scanned images (dust/debris on the glass or scanning lamp), scanner jams (for sheet-fed/document scanners), slow scanning speed.

Troubleshooting tips: clean the scanner glass and document feed rollers regularly; ensure the scanner driver/software is up to date and correctly installed; check and reseal cable connections (USB) or Wi-Fi settings for wireless scanners; restart the scanner and computer to clear temporary faults; run the manufacturer’s built-in diagnostic/calibration utility if scans appear misaligned or discolored.

Q: How does a paper (flatbed) scanner work? Functional diagram

1. The document is placed face-down on the glass platen.
2. A **light source (LED array or fluorescent lamp)** mounted on a moving carriage illuminates a thin strip of the document.
3. Light reflected from the document strip passes through a system of **mirrors and a lens**, focusing the reflected image onto a **CCD (Charge-Coupled Device) or CIS (Contact Image Sensor) array**.
4. The sensor array converts the varying light intensity (and color, via red/green/blue filters) at each point into a corresponding analog electrical signal.
5. An **Analog-to-Digital Converter (ADC)** converts this analog signal into digital pixel values.
6. The carriage steps forward slightly and repeats the process for the next strip, building up the complete image line-by-line until the whole document has been scanned, after which the digital image data is sent to the computer (via USB or network interface).

Q: Example of biometric technology

Examples include **fingerprint recognition**, **facial recognition**, **iris/retina scanning**, and **voice recognition** — each uses a unique physical or behavioral characteristic of a person to verify or identify their identity, commonly used in smartphone unlocking, security access control, and border/immigration control systems.

Q: Illustrate the Gantt chart for your project (CCE-322 Sessional)

This is a **project-specific** deliverable (your own Arduino/CCE-322 sessional project), so there's no universal "correct" answer — but structurally, a Gantt chart for a typical hardware/embedded sessional project would list tasks down the left column (e.g., "Literature review," "Component procurement," "Circuit design," "Coding/firmware development," "Hardware assembly," "Testing & debugging," "Report writing," "Final presentation") against a timeline (weeks) along the top, with horizontal bars showing each task's start week and duration, and some tasks overlapping (e.g., coding can start once circuit design is underway, while report writing typically runs in parallel across the later weeks). Fill in your own actual task list and durations based on your specific project's real schedule.

Q: CCE-322 Sessional/Arduino-specific: IOREF pin, Arduino IDE, ESP8266, digitalRead/digitalWrite tracing

- **IOREF pin:** present on newer Arduino boards, it provides a reference voltage matching the board's I/O operating voltage (e.g., 5V or 3.3V) so that shields/expansion boards can automatically detect and adapt to the correct logic-level voltage of the board they are plugged into, improving cross-compatibility between different Arduino board voltage standards.
- **Arduino IDE:** the software development environment used to write, compile, and upload sketches (programs written in a C/C++-based language) to Arduino boards; it includes a code editor, a compiler/toolchain, a serial monitor for debugging, and a library manager for adding external hardware/functionality support.
- **ESP8266:** a low-cost Wi-Fi module/system-on-chip that can be used with (or instead of) an Arduino to add wireless internet connectivity to a project, commonly used for IoT applications to send sensor data to the cloud or receive remote commands.
- **digitalRead()/digitalWrite() tracing:** `digitalWrite(pin, HIGH/LOW)` sets a specified digital output pin to a high (e.g., 5V/3.3V) or low (0V) voltage level, used to control outputs like LEDs, relays, or motor drivers; `digitalRead(pin)` reads the current logic-level state (HIGH or LOW) present on a specified digital input pin, used to read the state of buttons, switches, or digital sensor outputs. Tracing a program means stepping through the code line by line and noting, at each `digitalWrite/digitalRead` call, exactly which pin is affected and what logic level results, to verify the program's expected hardware behavior matches the intended design.

Final Notes

- Topics marked ★★ (Sections 1, 2, 3, 4, 5, 6, 7, 8, 9, 10) account for the bulk of marks historically and should be your first priority, followed by the ★★ topics (Sections 11–17), with the ★ topics (Section 18) reviewed last since they reflect the newer, shifting syllabus emphasis (IoT/WSN/real-time systems) seen in 2022–2023 papers.
- For numeric/design questions (memory interfacing, IEEE-754, flag calculations), the *method* shown above should be applied directly to whatever specific numbers appear on your exam paper — practice repeating each worked method with two or three different example values so you're comfortable adapting it under exam conditions.
- For diagram-based questions (LVDT construction, 8255/8279 pin/block diagrams, CRT operation, laser printer stages, DMA controller), practice redrawing each diagram from memory a few times — in most marking schemes, a clearly labelled diagram plus a short step-by-step explanation earns most of the available marks.